



ESCUELA DE  
INGENIERÍA EN CIENCIAS Y SISTEMAS  
FACULTAD DE INGENIERÍA  
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



**Día, Fecha:**

04/02/2026

**Hora de fin:**

7:00 pm

# Software Avanzado Sección B

Kevin Raúl Pozuelos Estrada



# Software Avanzado

---

Calificacion Practica 1



# Agenda

- HTML
- CSS
- JavaScript
- React JS
- Proyecto con Vite
- Backend
  - Nest JS
- Dudas
- Ejemplo
- Lectura del Proyecto Fase 1



# Frontend

Se centra en la presentación y la experiencia del usuario, asegurando que la interfaz sea atractiva, fácil de usar y responda de manera rápida a las acciones del usuario.

# HTML (HyperText Markup Language)

- Estructura básica de una página web.
- Etiquetas para organizar y mostrar contenido.

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi Primera Página</title>
</head>
<body>
  <h1>Hola, Mundo!</h1>
  <p>¡Bienvenido a mi página web!</p>
</body>
</html>
```

```
body {  
    font-family: 'Arial', sans-serif;  
    background-color: #f2f2f2;  
}  
  
h1 {  
    color: #333;  
}
```

# CSS (Cascading Style Sheets)

- Diseño y estilo de la página web.
- Controla la presentación y el formato.

# JavaScript

- Programación del lado del cliente.
- Interactividad y dinamismo en la página web.

```
function saludar() {  
    alert('¡Hola, visitante!');  
}
```



# Angular

Angular es un framework de JavaScript de tipo open source, es decir de código abierto, desarrollado por Google. Se utiliza en la creación y programación de aplicaciones web de una sola página. Como se explica en el sitio web de Angular, “los componentes son los bloques de construcción que componen una aplicación.

Se le saca mayor provecho si se hace uso de Typescript para el desarrollo

```
@Component({
  selector: 'app-viajeros',
  templateUrl: './viajeros.component.html',
  styleUrls: ['./viajeros.component.css'],
})
export class ViajerosComponent implements OnInit {
  @Input() viajero: Viajero;
  @Input() posicion: number;

  constructor() {}

  deleteViajero() {
  }

  editarViajero() {
  }

  ngOnInit(): void {}
}
```



# Vue JS

Se ha convertido en una opción popular para el desarrollo frontend debido a su enfoque declarativo, su capacidad para construir componentes reutilizables y su eficiente manejo del estado y la actualización del DOM.

```
function Video({ video }) {  
  return (  
    <div>  
      <Thumbnail video={video} />  
      <a href={video.url}>  
        <h3>{video.title}</h3>  
        <p>{video.description}</p>  
      </a>  
      <LikeButton video={video} />  
    </div>  
  );  
}
```

# React JS

Se ha convertido en una opción popular para el desarrollo frontend debido a su enfoque declarativo, su capacidad para construir componentes reutilizables y su eficiente manejo del estado y la actualización del DOM.

```
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6
7  function HelloWorld() {
8    return <h1 className="greeting">Hello, world!</h1>
9  }
10
11  const root = ReactDOM.createRoot(document.getElementById('root'));
12  root.render(
13    <React.StrictMode>
14      <HelloWorld />
15    </React.StrictMode>
16  );
17
18  // If you want to start measuring performance in your app, pass a function
19  // to log results (for example: reportWebVitals(console.log))
20  // or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
21  reportWebVitals();
```

# Principales Características de React

Componentes

JSX

Virtual DOM

Estado y  
Propiedades

React Router

Gestión de  
Estado

React Hooks

# React con vite



```
npx create-vite my-react-app --template react  
cd my-react-app
```

Primero, asegúrate de tener Node.js instalado en tu máquina. Luego, crea un nuevo proyecto con Vite usando la plantilla de React

## Crear un Proyecto con Vite y React

# Estructura del Proyecto

---

```
my-react-app
|-- node_modules
|-- public
|-- src
|   |-- components
|   |-- App.css
|   |-- App.jsx
|   |-- index.css
|   |-- main.jsx
|-- .gitignore
|-- index.html
|-- package.json
|-- README.md
|-- vite.config.js
```



```
npm install
```

```
npm run dev
```

Desarrollo Local

# Crear Componentes de React

```
// src/components/MiComponente.jsx
import React from 'react';

const MiComponente = () => {
  return (
    <div>
      <h2>Mi Componente React</h2>
      <p>Hola desde Vite y React!</p>
    </div>
  );
};

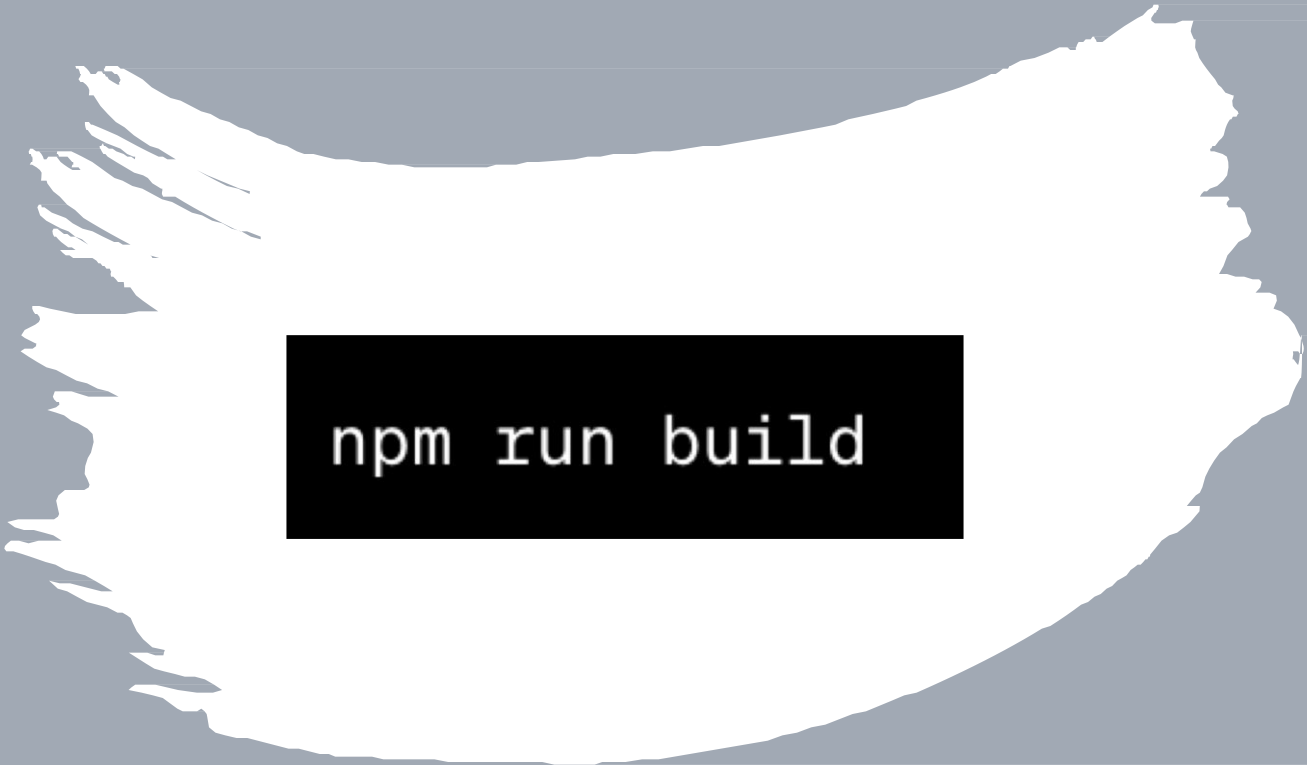
export default MiComponente;
```

# Uso de Componentes en App.jsx

```
// src/App.jsx
import React from 'react';
import './App.css';
import MiComponente from './components/MiComponente';

function App() {
  return (
    <div className="App">
      <h1>Mi Aplicación React con Vite</h1>
      <MiComponente />
    </div>
  );
}

export default App;
```



```
npm run build
```

# Compilar y Construir

# Ventajas de crear el proyecto con Vite

---

Arranque Rápido

---

Desarrollo en Tiempo Real

---

Soporte Nativo para

---

TypeScript Optimización para

---

Producción Configuración

---

Simple

---

Múltiples Entradas (Multipage)

---

Esquema de Rutas

---

Integrado Soporte para Vue  
y React Ecosistema

# Documentación



1

<https://react.dev/>

2

<https://vitejs.dev/guide/>

3

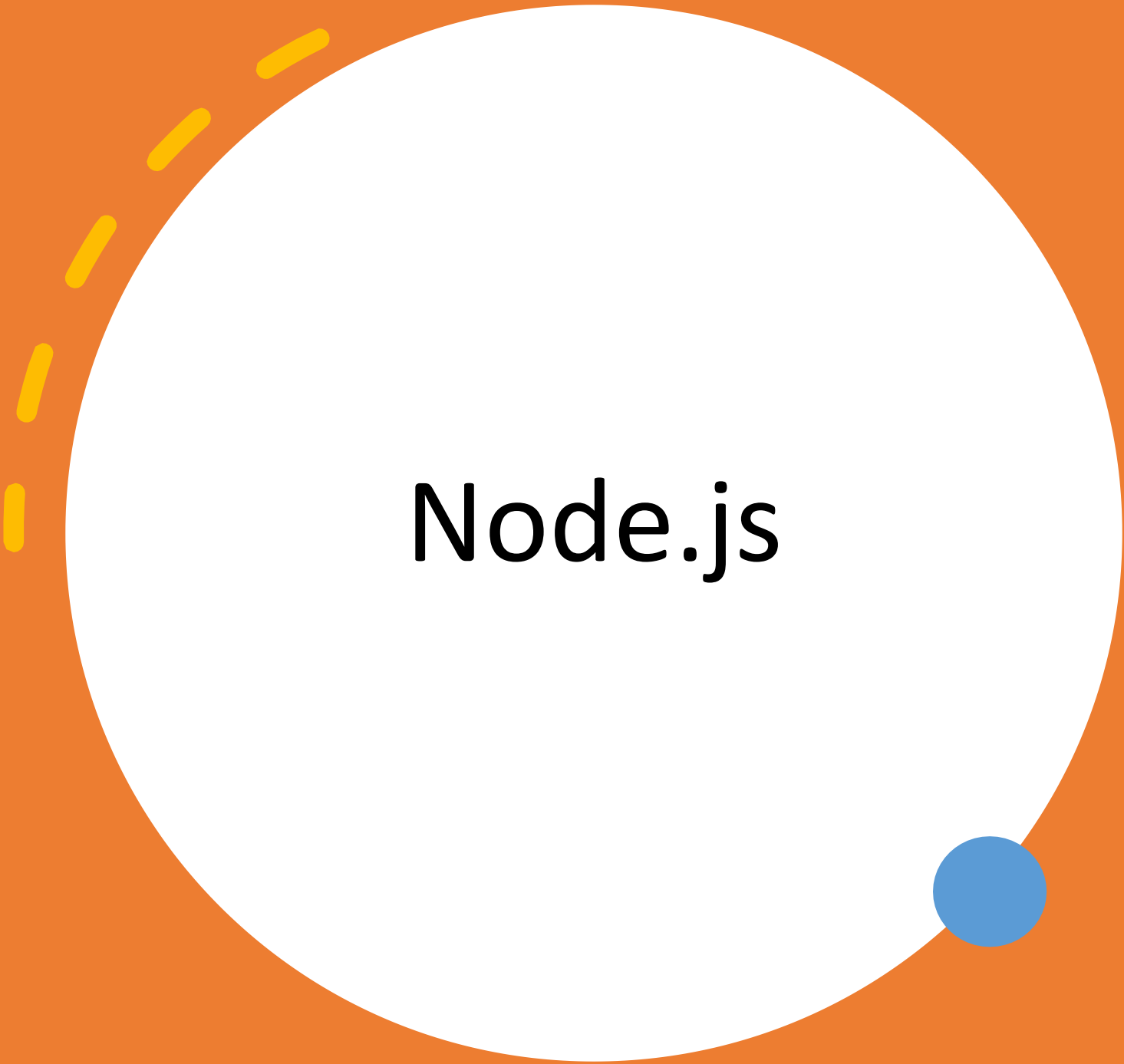
<https://mui.com/material-ui/getting-started/>

4

<https://react-bootstrap.netlify.app/docs/getting-started/introduction/>

# Backend

Es responsable de la funcionalidad interna y la gestión de datos de una aplicación, proporcionando la infraestructura necesaria para que el frontend pueda operar y ofrecer una experiencia completa al usuario.

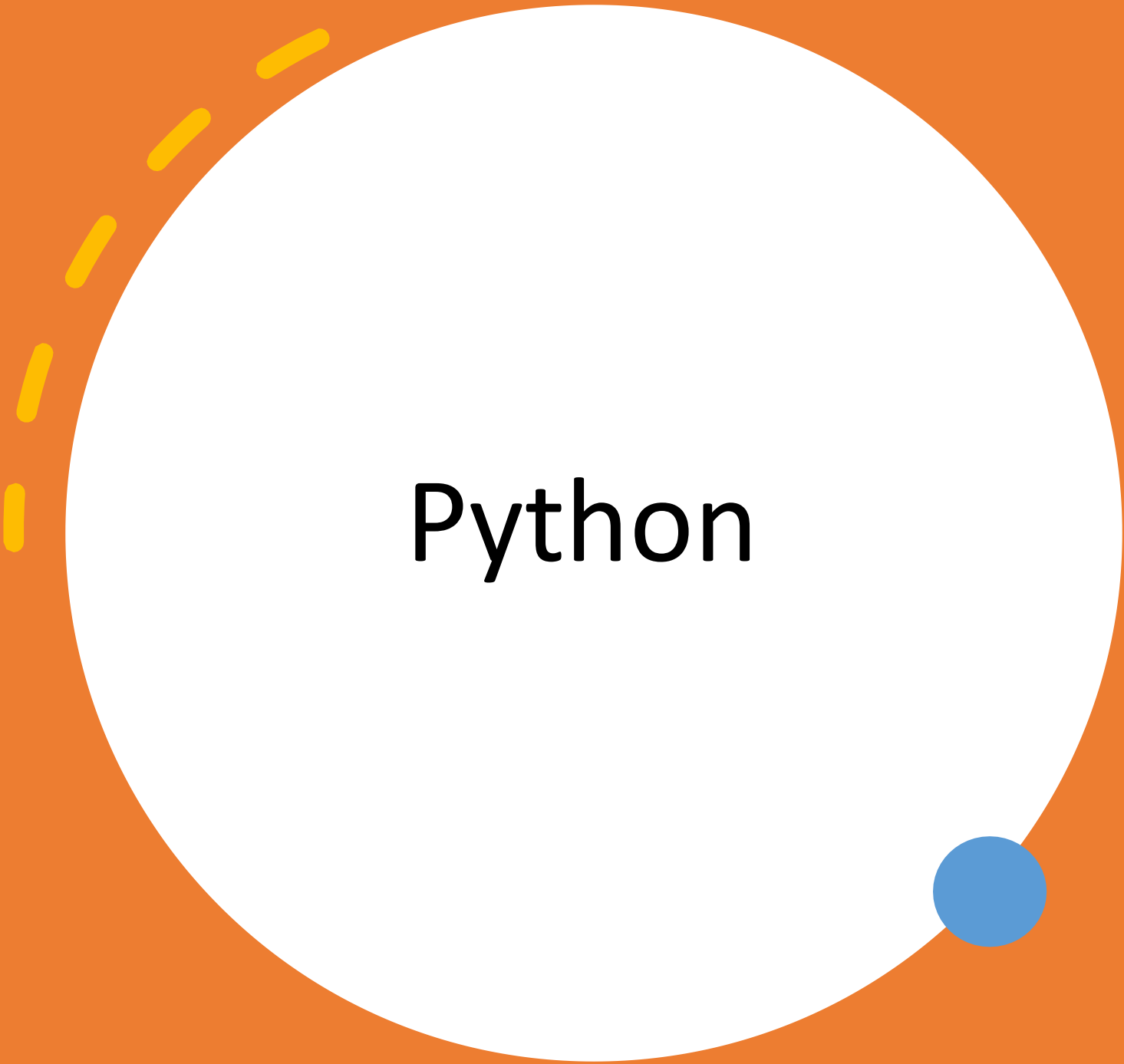
The image features the Node.js logo, which consists of a large white circle centered on an orange background. Inside the white circle, the text "Node.js" is written in a black, sans-serif font. To the left of the white circle, there is a dashed yellow arc. At the bottom right of the white circle, there is a small solid blue circle.

Node.js



# Características

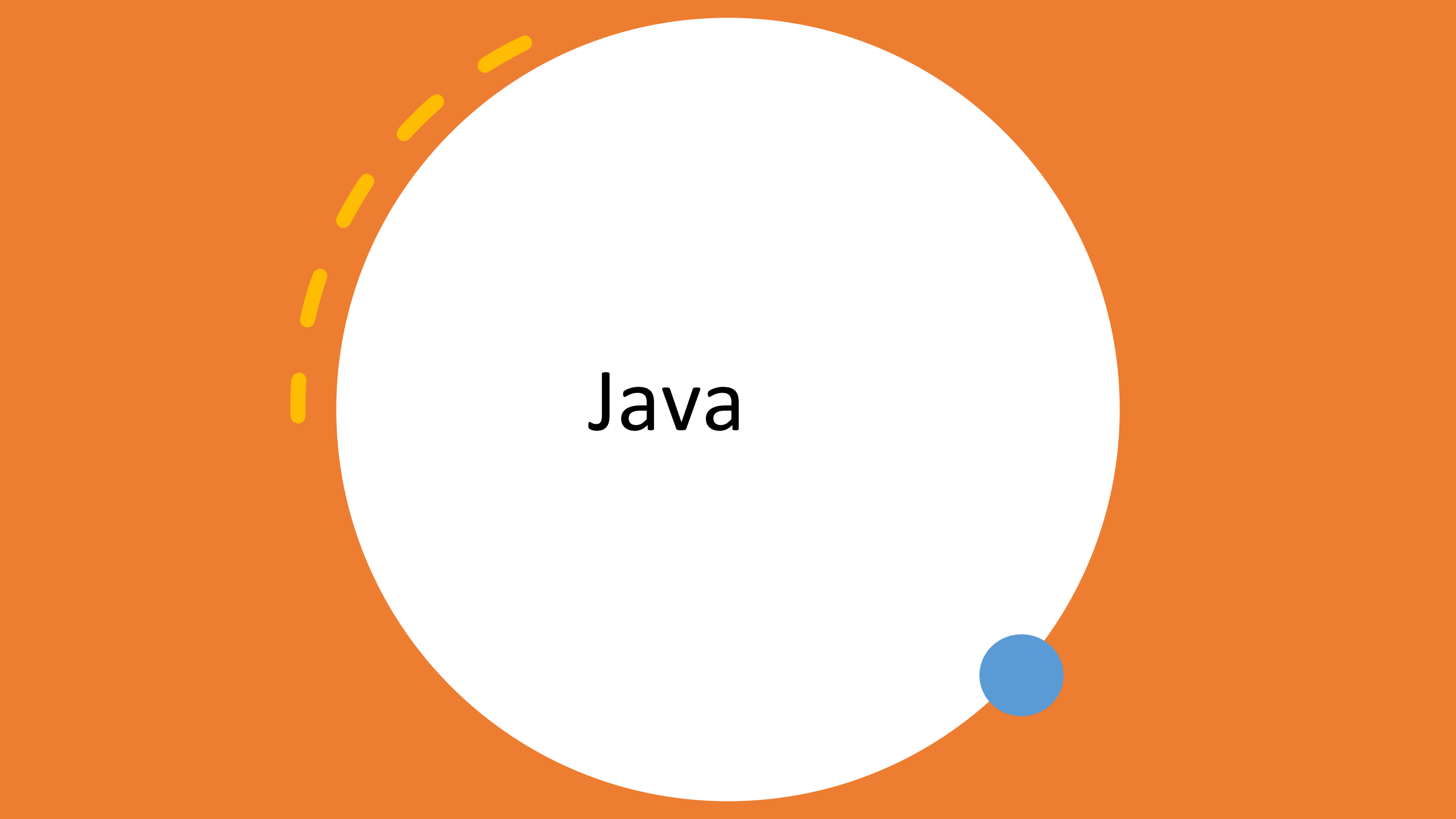
- Motor V8 de Google Chrome
- JavaScript del Lado del Servidor
- Manejo Asíncrono y Eventos
- Módulos y manejadores de paquetes
- Framework Express para API REST
- Streaming de Datos
- Comunidad Activa

The image features a large white circle centered on an orange background. The word "Python" is written in black, sans-serif font in the center of the white circle. To the left of the circle, there is a dashed yellow arc. At the bottom right of the circle, there is a small solid blue circle.

Python

# Características

- Es un lenguaje interpretado
- Orientado a objetos, pero flexible a otros tipos de programación
- Lenguaje por excelencia para desarrollos de IA y análisis de datos
- Gran variedad de librerías
- No recomendado para velocidad de transacción y optimización
- Comunidad Activa



Java

# Características

- Lenguaje compilado
- Al igual que python, multiplataforma
- Orientado a objetos completamente
- Complejidad intermedia
- Seguridad elevada y cantidad decente de librerías
- Una de las comunidades más antiguas y activas

# Bases de datos

Las bases de datos son conjuntos organizados de información o datos que se almacenan y gestionan en sistemas informáticos. Estas permiten guardar, recuperar y manipular datos de manera eficiente. Se utilizan en una amplia variedad de aplicaciones, desde sitios web y aplicaciones móviles hasta sistemas empresariales complejos.

# Documentación



1

**Bases de datos relacionales:** Organizan los datos en tablas con filas y columnas. Utilizan el lenguaje SQL (Structured Query Language) para gestionar los datos. Ejemplos: MySQL, PostgreSQL, Oracle, SQL Server.

2

**Bases de datos NoSQL:** Se diseñaron para manejar grandes volúmenes de datos no estructurados. Incluyen varios tipos:

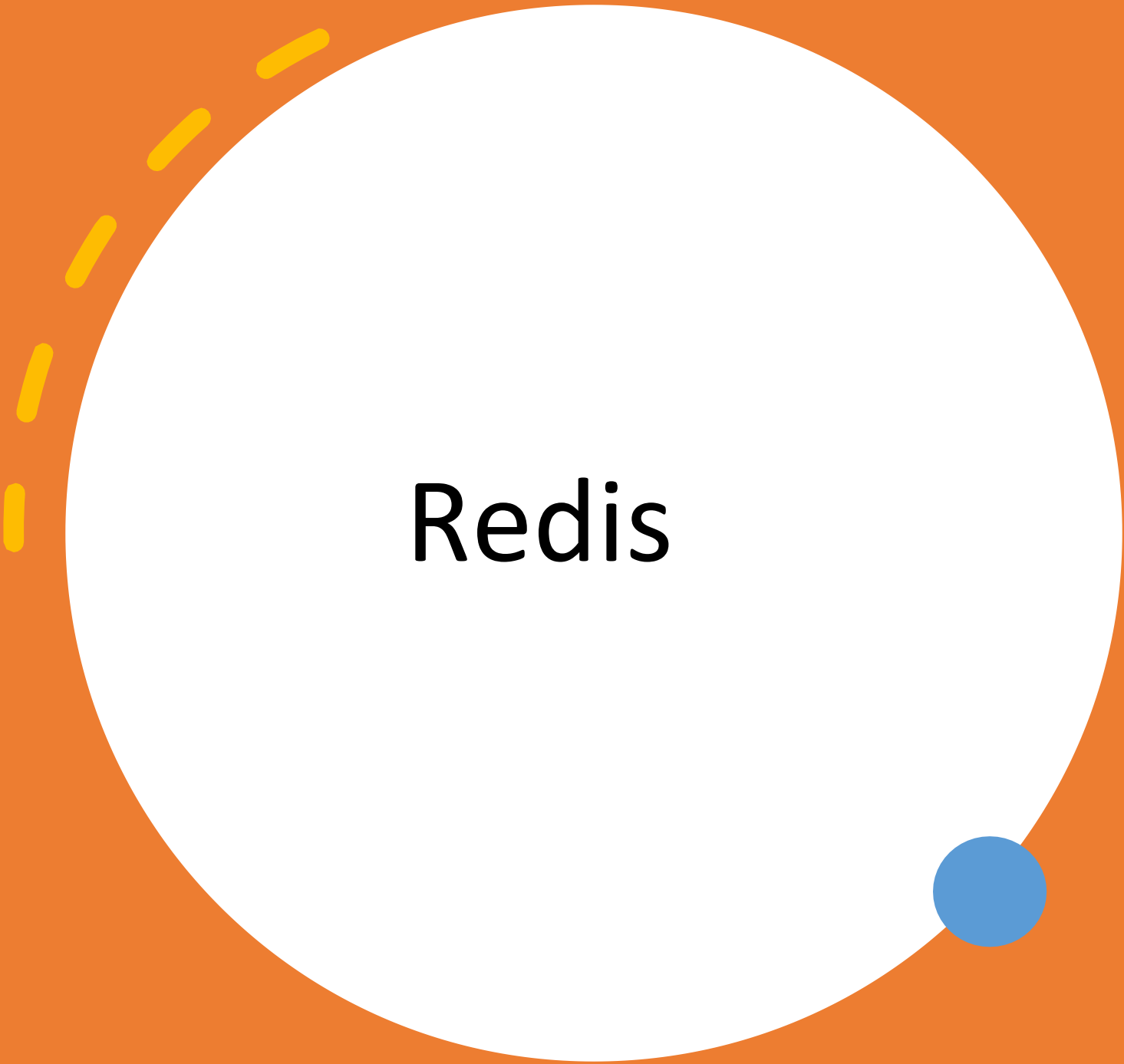
- **Documentales:** JSON o BSON (Ej: MongoDB).
- **Clave-Valor:** Datos como pares de clave y valor (Ej: Redis).
- **Columnar:** Optimizadas para grandes volúmenes de datos distribuidos (Ej: Cassandra).
- **Grafos:** Especializadas en relaciones complejas entre datos (Ej: Neo4j).

3

**Bases de datos en la nube auto administradas:** Almacenadas y gestionadas en plataformas en la nube (Ej: Amazon RDS, Google Cloud Spanner).

4

**Bases de datos orientadas a objetos:** Almacenan datos en forma de objetos, como en la programación orientada a objetos (Ej: db4o).

The Redis logo is centered on a solid orange background. It consists of a large white circle. Inside this circle, the word "Redis" is written in a black, sans-serif font. To the left of the white circle, there is a dashed yellow arc. At the bottom right of the white circle, there is a small solid blue circle.

Redis



# Características

Almacenamiento en memoria: Redis guarda todos los datos en la memoria RAM, lo que permite un acceso extremadamente rápido. También puede persistir datos en disco para respaldos y recuperación.

Estructuras de datos avanzadas: A diferencia de otros almacenes clave-valor, Redis admite estructuras de datos complejas, como:

- Strings: Cadenas de texto o datos binarios.
- Lists: Listas enlazadas de elementos ordenados.
- Sets: Conjuntos desordenados de elementos únicos.
- Sorted Sets: Conjuntos ordenados por un valor de puntuación.
- Hashes: Colecciones de pares clave-valor similares a objetos JSON.
- Streams: Para manejar flujos de datos en tiempo real.

Persistencia opcional: Aunque es una base de datos en memoria, Redis ofrece opciones de persistencia en disco mediante el mecanismo RDB (Redis Database Backup) o AOF (Append Only File).

Réplica y alta disponibilidad: Soporta replicación maestro-esclavo y alta disponibilidad mediante Redis Sentinel.

Escalabilidad: Se puede usar en un entorno distribuido con Redis Cluster, lo que permite dividir datos entre múltiples nodos.

# Autenticacion y Autorizacion

Que son?

# Autenticación

Es el proceso de verificar la identidad de un usuario o sistema. Se trata de confirmar que alguien es quien dice ser.

Ejemplos comunes:

- Ingresar un nombre de usuario y contraseña.
- Utilizar huellas dactilares, reconocimiento facial o autenticación de dos factores (2FA).
- Iniciar sesión con una cuenta de Google o Facebook en otras aplicaciones.

# Autorización

Es el proceso de determinar los permisos que tiene un usuario autenticado. Una vez que se verifica su identidad, se define qué acciones puede realizar o qué recursos puede acceder.

Ejemplos comunes:

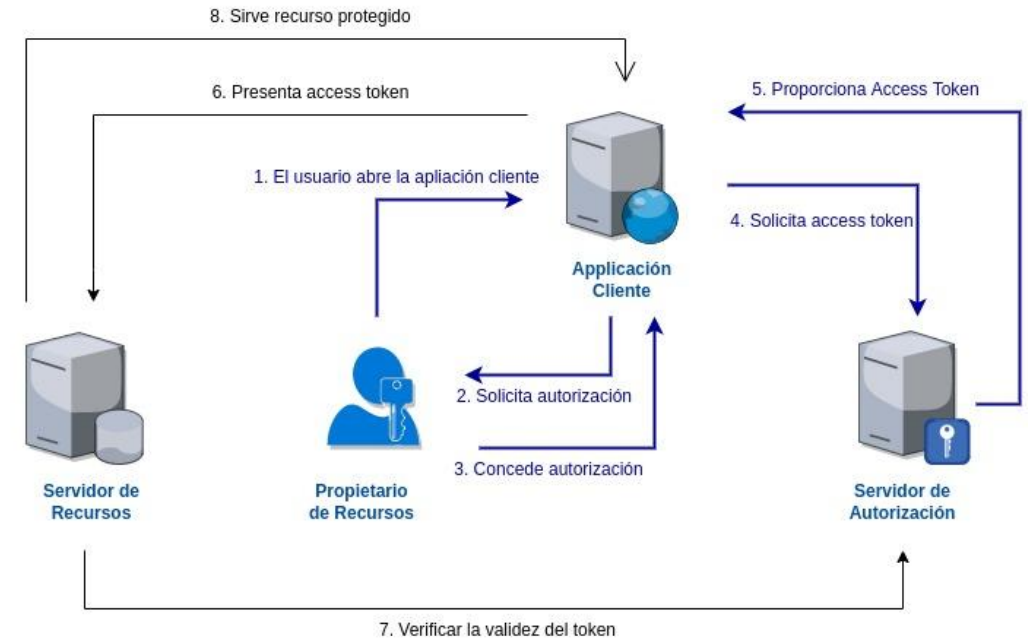
- Acceder a diferentes secciones de una aplicación según el rol (ej. administrador, usuario normal, invitado).
- Permitir o denegar la modificación de datos.
- Controlar el acceso a archivos o funcionalidades en una plataforma.

# Oauth 2.0

OAuth 2.0 es un protocolo de autorización que permite a las aplicaciones acceder a recursos de usuario en otro servicio sin necesidad de exponer sus credenciales (como la contraseña). Es ampliamente utilizado por plataformas como Google, Facebook y GitHub para permitir el inicio de sesión con una cuenta externa o para conceder permisos a aplicaciones de terceros.

# Como funciona Oauth?

1. Resource Owner (Propietario del Recurso): El usuario que concede acceso a sus datos.
2. Client (Cliente): La aplicación que quiere acceder a los datos.
3. Resource Server (Servidor de Recursos): La API que protege los datos (ej. API de Google).
4. Authorization Server (Servidor de Autorización): Verifica la identidad del usuario y emite tokens de acceso.

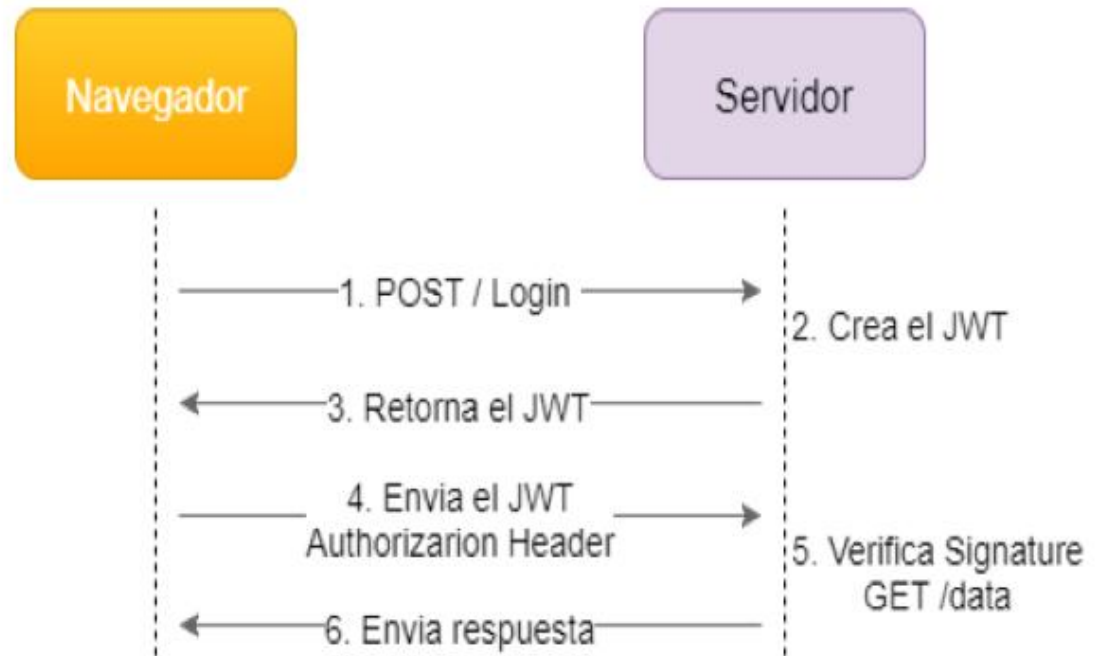


# JWT

Método popular de autenticación en aplicaciones web y móviles. Se utiliza para verificar la identidad del usuario y, en algunos casos, también para la autorización.

Es un token en formato JSON que se utiliza para transmitir información de forma segura y compacta entre dos partes (por ejemplo, un cliente y un servidor). Se firma digitalmente para garantizar su integridad y autenticidad.

# Cómo funciona JWT?



---

¿Dudas?

---



¡GRACIAS POR SU  
ATENCIÓN!

DUDAS

